Tutorial 1B (to be completed in week1)
Data Modelling (Part 1)

Tutorial Objectives:

- Introduce Visio
- Draw a conceptual model using Visio

This tutorial is prepared using Microsoft Visio 2019. Visio can be accessed from https://remote.massey.ac.nz. However, we must register each one of you individually to access. So, in case you cannot access because of late enrolment get in touch with Indu (i.b.sofat@massey.ca.nz – use the appropriate subject line)

Visio is another modelling tool that is being used in this course for drawing purposes (e.g., to draw conceptual model). Your textbook also uses Visio to create database designs using mainly Crow's Foot Database notation.

Note(s):
This tutorial is provided to help you with using Visio; as an alternative to the JDeveloper tutorials.

Tutorial Tasks:

a. **Draw a conceptual model for the given case study using an ERD (Entity Relationship Diagram)**
b. **Represent the ERD as a Database Model Diagram using Visio**
c. **Extend the ERD (add attributes, etc.)**

PART a

a. Draw a Conceptual model for the given case study

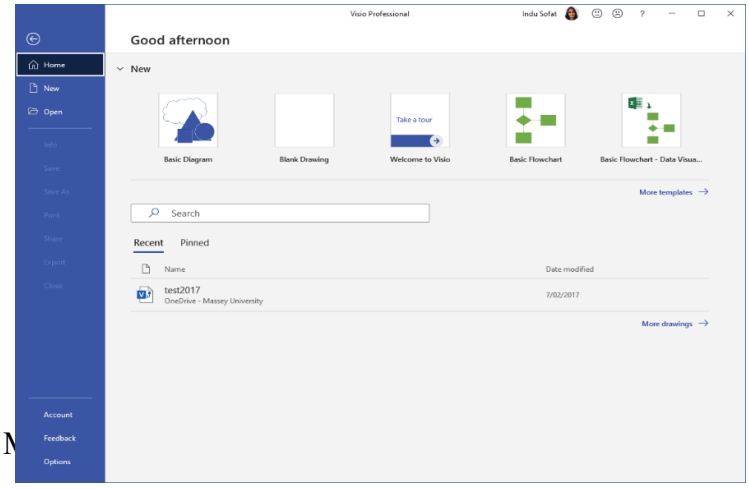
Read the Tiny College case study* on pg. 57-58, ch2 (Ed13) to identify the main entities and relationships.
Represent these entities and relationships as an ERD (hand-drawn is fine at this stage).

PART b

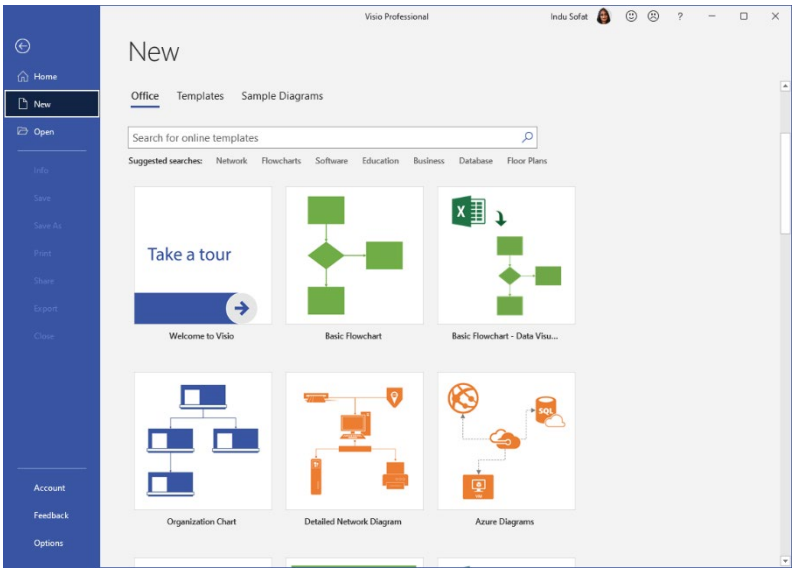
b. Represent the ERD as a Database Model Diagram using Visio

VMware Horizon Client was used to access Visio (2019).

As there are many templates, select More templates.

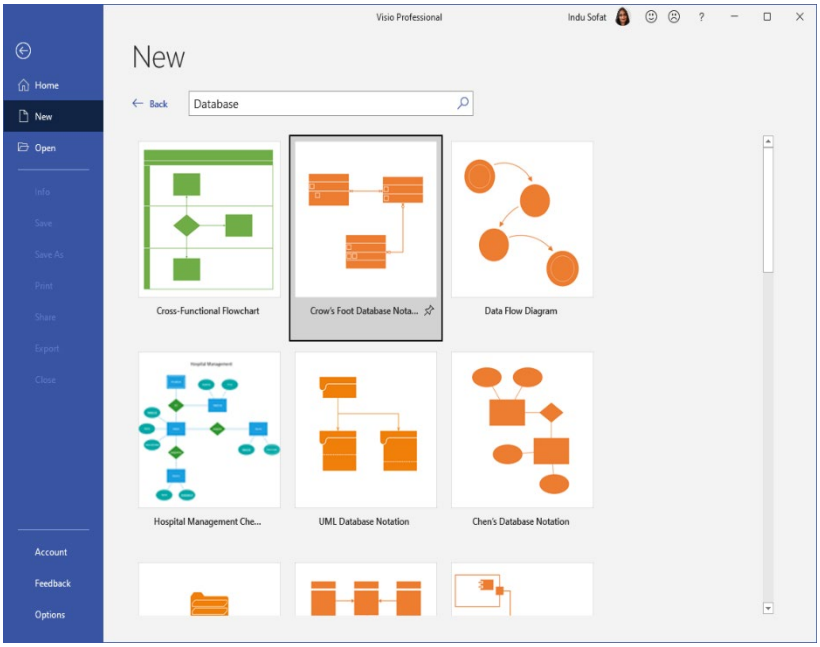


Select Database tab

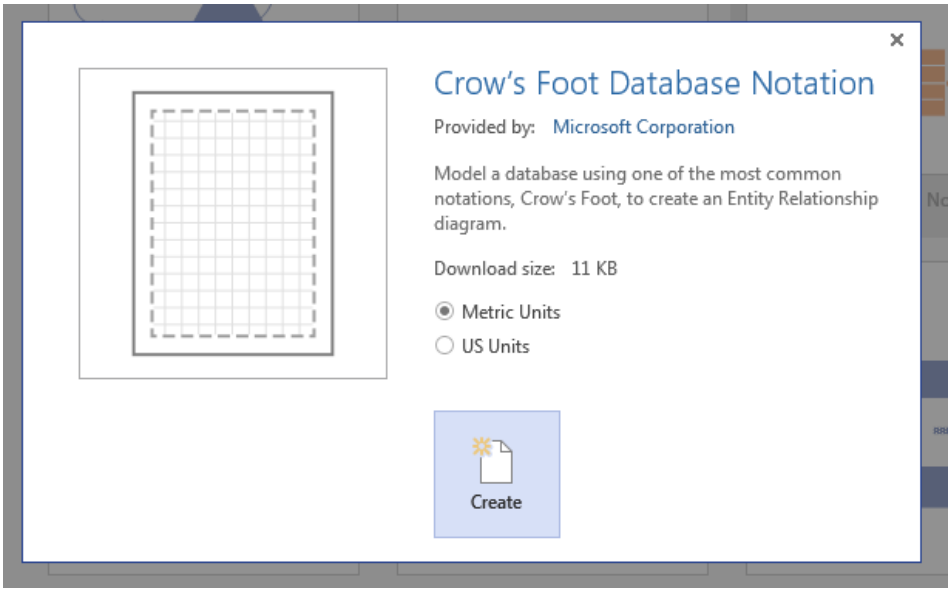


Select Crow's Foot Database Notation from the Featured diagrams.

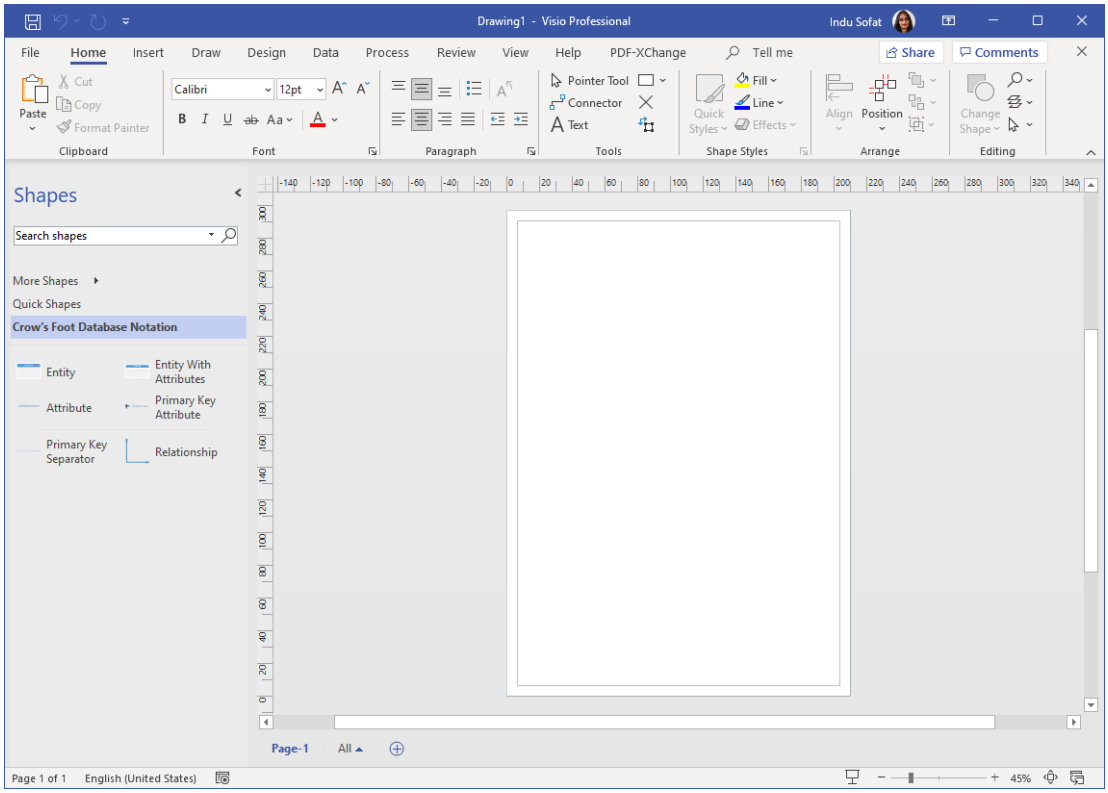
You can also select the diagram from the given set of Suggested searches (e.g., Network, Flowcharts, **Software**, etc.)



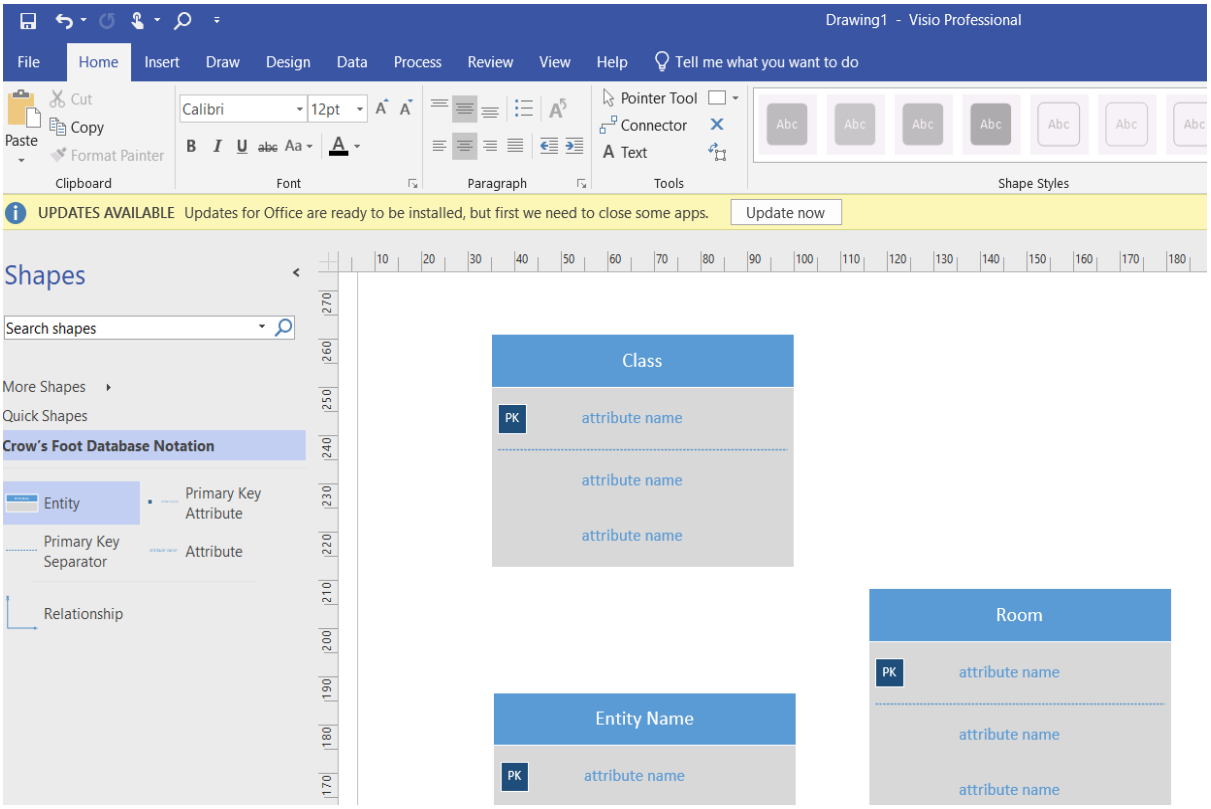
After, selecting Crow's Foot Database Notation from the displayed set of notations, click Create.



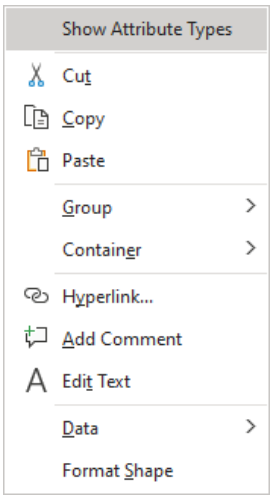
You are now provided with a blank page along with necessary database elements to draw your ERD.



Add entities by dragging and dropping on the drawing palette. Select the dragged entity from the drawing palette and either double click or r-click to change its name, etc.

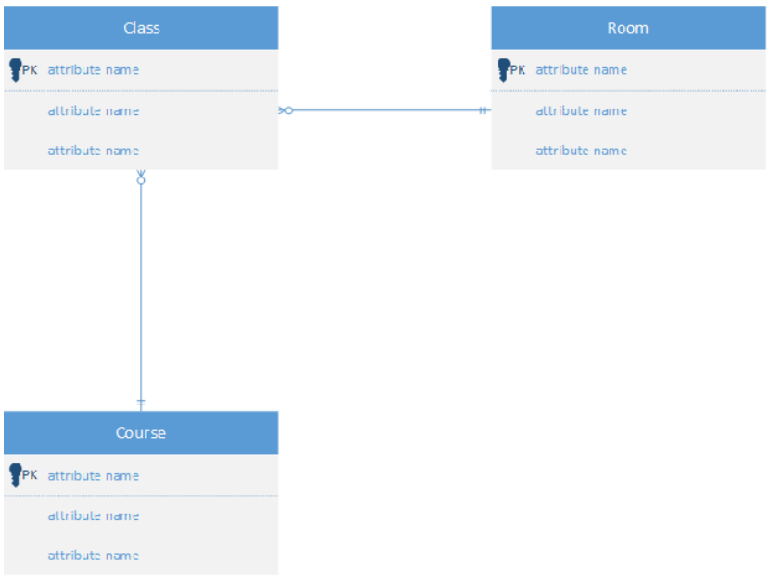


When you r-click you get the context related menu. For example, while selecting a table you can select various options (e.g., Edit Text – to change the table name, Show Attribute Types, etc.) to do different things. Explore these options.



Keep working on this diagram until all the required entities (Room, etc.) are added. Save your work with the file name such as TinyCollegeConceptual.vsdx

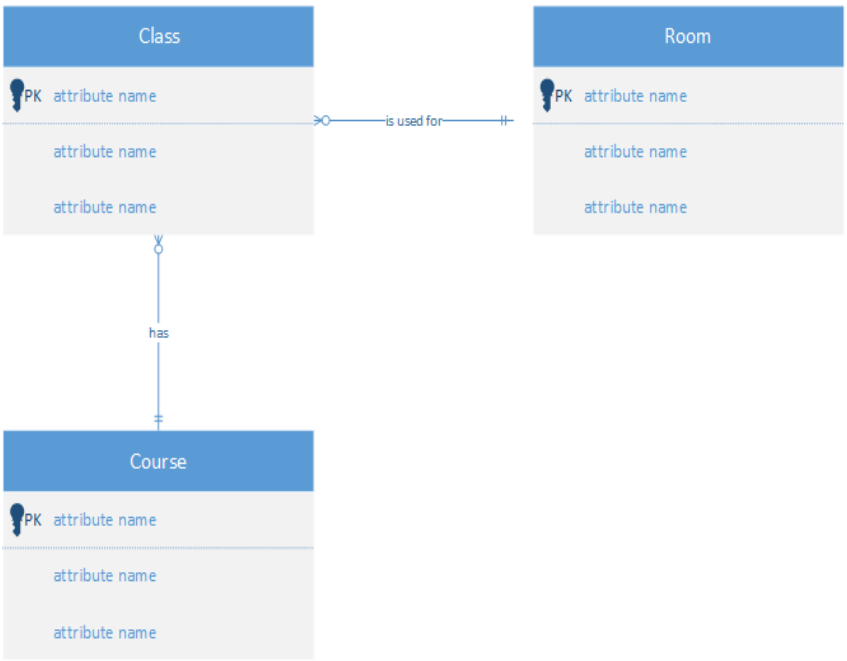
Similarly, add associations by dragging and dropping Relationship symbol between the appropriate entities (e.g., Course – Class). The relationship joining should be done properly – the whole entity (boundary) goes green when the relationship is joined properly on that entity's end. Also, join the one and many ends appropriately (e.g., relationship between Course and Class is one to many - Each Course can have many Classes i.e., the multiplicity is shown one at the Course end and many at the Class end).



Double-click or r-click the relationship to enter the relationship name (e.g., is used for, has, generates, etc.).
Remember to read relationships from the one side:

- A Course has many Classes.
- A Room is used for many Classes.

(**Note:** The diagram shows only a small section of the ERD.)



PART c

c. Extend the ERD (add attributes, etc.)

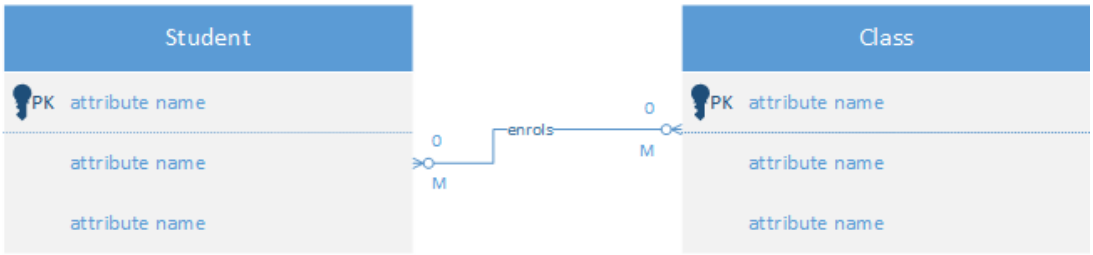
Add some of the known attributes to all the entities (e.g., student ID, student first name, course title, etc.).

Before printing your ERD, consider various options (e.g., Orientation, etc.).

Notes:

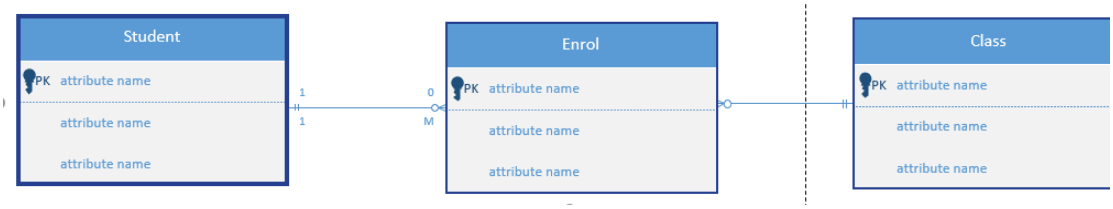
1. Confirm (either with the client or in client's absence list under assumptions) the ambiguous business rules (these rules translate to multiplicities) before you put them on the diagram. Thinking on the lines of following situations is helpful.
   - What if a Professor is on sabbatical leave or is new and therefore, not yet assigned to any Class?
   - What if a Class is taught by more than one Professor?
   - Are we interested in storing data about non-teaching rooms as well?
   - Do we want to allow students to be part of the database even if they have not enrolled for any Course(s) yet?

2. Although a conceptual diagram (technology and implementation independent) may have many-to-many (M:N) relationships but we encourage you to think about the intersection entities right from the start (relational models do not support M:M) and therefore resolve any M:N relationships by having two 1:M relationships. For example, for a M:N relationship between the entities Student and Class, think about an intersection entity (e.g. Enrol).



Avoid M:N relationship between the entities Student and Class.

The diagram below now includes an intersection entity Enrol between the Student and Class entities and has two 1:M relationships.



***Screen Dump of the Tiny College Case Study (pg 57-58)**

- A PROFESSOR may teach many CLASSes, and each CLASS is taught by only one PROFESSOR; there is a 1:M relationship between PROFESSOR and CLASS.
- A CLASS may ENROLL many students, and each STUDENT may ENROLL in many CLASSes, thus creating an M:N relationship between STUDENT and CLASS. (You will learn about the precise nature of the ENROLL entity in Chapter 4.)
- Each COURSE may generate many CLASSes, but each CLASS references a single COURSE. For example, there may be several classes (sections) of a database course that have a course code of CIS-420. One of those classes might be offered on MWF from 8:00 a.m. to 8:50 a.m., another might be offered on MWF from 1:00 p.m. to 1:50 p.m., while a third might be offered on Thursdays from 6:00 p.m. to 8:40 p.m. Yet, all three classes have the course code CIS-420.
- Finally, a CLASS requires one ROOM, but a ROOM may be scheduled for many CLASSes. That is, each classroom may be used for several classes: one at 9:00 a.m., one at 11:00 a.m., and one at 1:00 p.m., for example. In other words, there is a 1:M relationship between ROOM and CLASS.